



Módulo 1

Introducción a WPF



1- Introducción

Definición

Windows Presentation Foundation (WPF) es un sistema de presentación para crear aplicaciones cliente de Windows que proporcionen una experiencia impactante para el usuario desde el punto de vista visual. Con WPF, puede crear una amplia gama de aplicaciones independientes y hospedadas en explorador.

El núcleo de WPF es un motor de representación independiente de la resolución y basado en vectores construido para aprovechar al máximo el hardware de gráficos moderno. WPF amplía el núcleo con un completo conjunto de características de programación de aplicaciones, entre las que se incluyen Lenguaje de marcado de aplicaciones extensible (XAML), controles, enlace de datos, diseño, gráficos 2D y 3D, animación, estilos, plantillas, documentos, multimedia, texto y tipografía. WPF se incluye en Microsoft .NET Framework, lo que permite crear aplicaciones que incorporen otros elementos de la biblioteca de clases de .NET Framework.

Programar con WPF

WPF constituye un subconjunto de tipos de .NET Framework, en su mayoría ubicados en el espacio de nombres de **System.Windows**. Si ha creado previamente aplicaciones con .NET Framework mediante tecnologías administradas como ASP.NET y formularios Windows Forms, los conceptos fundamentales de la programación en WPF deberán resultarle familiares; creará instancias de clases, definirá propiedades, llamará a métodos y controlará eventos con el lenguaje de programación de .NET Framework que prefiera, como C# o Visual Basic.

Para admitir algunas de las funciones de WPF más eficaces y simplificar la experiencia de programación, WPF incluye construcciones de programación adicionales que mejoran las propiedades y los eventos

Código de lenguaje marcado y código subyacente

WPF proporciona mejoras de programación adicionales para el desarrollo de aplicaciones cliente de Windows. Una mejora evidente es la capacidad para programar una aplicación mediante código de lenguaje marcado y subyacente, una experiencia con la que resultará familiar a los programadores de ASP.NET. En general, se utiliza el lenguaje marcado Lenguaje de marcado de aplicaciones extensible (**XAML**) para implementar la apariencia de una aplicación, y los lenguajes de programación administrados (c#, visual Basic, etc.) para implementar su comportamiento. Esta separación entre la apariencia y el comportamiento aporta las ventajas siguientes:

- Se reducen los costos de programación y mantenimiento, al no estar el marcado específico de la apariencia estrechamente relacionado con el código específico del comportamiento.
- La programación es más eficaz porque los diseñadores pueden implementar la apariencia de una aplicación al mismo tiempo que los programadores implementan su comportamiento.
- Se pueden utilizar varias herramientas de diseño para implementar y compartir el lenguaje marcado XAML, a fin de responder a los requisitos de los colaboradores de programación de aplicaciones. *Microsoft Expression Blend* proporciona una experiencia apropiada para los diseñadores, mientras que *Visual Studio 2008* está dirigido a los programadores.
- La globalización y localización de las aplicaciones WPF se ha simplificado en gran medida.

XAML

XAML es un lenguaje de marcado basado en XML que se utiliza para implementar la apariencia de una aplicación mediante declaración. Se suele utilizar para crear ventanas, cuadros de



diálogo, páginas y controles de usuario, así como para rellenarlos con controles, formas y gráficos.

Ejemplo:

```
<Window
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    Title="Ventana con un Botón"
    Width="250" Height="100">

    <Button Name="button">Presionar</Button>

</Window>
```

El código XAML define una ventana y un botón mediante los elementos *Window* y *Button*, respectivamente. Cada elemento se configura con atributos o propiedades, como el atributo *Title* del elemento *Window*, para especificar el texto de la barra de título de la ventana. En tiempo de ejecución, WPF convierte los elementos y los atributos definidos en el marcado en instancias de clases de WPF.

Puesto que XAML se basa en XML, la interfaz de usuario que se crea con este lenguaje se ensambla en una jerarquía de elementos anidados que se denomina árbol de elementos. El árbol de elementos proporciona una manera lógica e intuitiva de crear y administrar las interfaces de usuario.

Código Subyacente

El comportamiento principal de una aplicación es implementar la funcionalidad que responde a las interacciones con el usuario, lo que incluye controlar los eventos (por ejemplo, hacer clic en un menú, una barra de herramientas o un botón) y llamar, en respuesta, a la lógica de negocios y de acceso a los datos. En WPF, este comportamiento se suele implementar en código asociado al marcado. Este tipo de código se denomina subyacente.

En el ejemplo siguiente se muestran el código subyacente y el marcado actualizado del ejemplo anterior.

```
<Window
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    Title="Ventana con un Botón"
    Width="250" Height="100">

    <Button Name="button" Click="button_Click">Presionar</Button>

</Window>
```

Código subyacente:

```
using System.Windows;
```

```
namespace EjemploWPF
```

```
{
    public partial class miEjemplo : Window
    {
        public void button_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show("Probando Windows Presentation Foundation!");
        }
    }
}
```



Aplicaciones WPF

Microsoft .NET Framework, System.Windows, así como el código de lenguaje marcado y subyacente, constituyen la base de la experiencia de programación de aplicaciones en WPF. Además, WPF cuenta con características completas para crear experiencias de usuario con contenido enriquecido. Para empaquetar este contenido y distribuirlo a los usuarios en forma de aplicaciones, WPF proporciona tipos y servicios denominados colectivamente el modelo de aplicación. El modelo de aplicación admite la programación de aplicaciones independientes y hospedadas en explorador.

- Aplicaciones independientes: Para las aplicaciones independientes, puede utilizar la clase *Window* para crear las ventanas y cuadros de diálogo a los que se tiene acceso desde las barras de menús y las barras de herramientas.
- Aplicaciones hospedadas en explorador: También denominadas Aplicaciones del explorador XAML (XBAPs – XAML Browsers Applications, generan archivos con extensión xbp), puede crear páginas (Page) y funciones de página (PageFunction<T>) entre las que se puede navegar mediante hipervínculos (clases Hyperlink). Estas aplicaciones pueden hospedarse en Internet Explorer a partir de la versión 6.

Seguridad

Dado que las XBAPs se hospedan en un explorador, la seguridad es importante. En particular, las XBAPs utilizan un recinto de seguridad de confianza parcial para aplicar restricciones menores o iguales a las que se imponen a las aplicaciones basadas en HTML. Aún así, la mayoría de las características de WPF se pueden ejecutar con seguridad desde las XBAPs.

2- Conceptos Básicos

Controles

En WPF, un control es un término general que se aplica a una categoría de clases de WPF hospedadas en una ventana o una página, tienen una interfaz de usuario (UI) e implementa un comportamiento determinado.

Los controles casi siempre detectan las acciones del usuario y responden a ellas. El sistema de entrada de WPF utiliza eventos directos y enrutados para admitir entradas de texto, la administración del enfoque y la posición del mouse.

A continuación se muestra la lista de controles de WPF integrados.

- Botones: Button y RepeatButton.
- Cuadros de diálogo: OpenFileDialog, PrintDialog y SaveFileDialog.
- Entradas manuscritas digitales: InkCanvas y InkPresenter.
- Documentos: DocumentViewer, FlowDocumentPageViewer, FlowDocumentReader, FlowDocumentScrollViewer y StickyNoteControl.
- Entrada: TextBox, RichTextBox y PasswordBox.
- Diseño: Border, BulletDecorator, Canvas, DockPanel, Expander, Grid, GridView, GridSplitter, GroupBox, Panel, ResizeGrip, Separator, ScrollBar, ScrollViewer, StackPanel, Thumb, Viewbox, VirtualizingStackPanel, Window y WrapPanel.
- Multimedia: Image, MediaElement y SoundPlayerAction.
- Menús: ContextMenu, Menu y ToolBar.
- Navegación: Frame, Hyperlink, Page, NavigationWindow y TabControl.
- Selección: CheckBox, ComboBox, ListBox, TreeView y RadioButton, Slider.
- Información para el usuario: AccessText, Label, Popup, ProgressBar, StatusBar, TextBlock y ToolTip.

Diseño



Al crear una interfaz de usuario, se organizan los controles según su ubicación y tamaño para crear un diseño. Un requisito fundamental de cualquier diseño es adaptarse a los cambios de tamaño de la ventana y de configuración de pantalla. En lugar de obligarle a escribir código que adapte el diseño en estas circunstancias, WPF le proporciona un sistema de diseño extensible de primera clase.

La base del sistema de diseño es la situación relativa, que aumenta la capacidad de adaptación a los cambios en la configuración de las ventanas y de la pantalla. Además, el sistema de diseño administra la negociación entre los controles para determinar el diseño. La negociación es un proceso de dos pasos: en primer lugar, el control indica a su elemento primario qué ubicación y tamaño necesita; en segundo lugar, el elemento primario indica al control de qué espacio dispone.

El sistema de diseño se expone a los controles secundarios mediante las clases base de WPF. Para los diseños comunes, como son las cuadrículas, el apilamiento y el acoplamiento, WPF incluye varios controles de diseño:

- Canvas: los controles secundarios proporcionan su propio diseño.
- DockPanel: los controles secundarios se alinean con los bordes del panel.
- Grid: los controles secundarios se sitúan por filas y columnas.
- StackPanel: los controles secundarios se apilan vertical u horizontalmente.
- VirtualizingStackPanel: los controles secundarios se organizan en una vista "virtual" de una sola línea en sentido horizontal o vertical.
- WrapPanel: los controles secundarios se sitúan por orden de izquierda a derecha y se ajustan a la línea siguiente cuando hay más controles de los que caben en la línea actual.

Enlace de Datos

La mayoría de las aplicaciones se crean para proporcionar recursos a los usuarios que les permitan ver y editar los datos. Para aplicaciones WPF, el trabajo de almacenar los datos y tener acceso a ellos se proporciona mediante tecnologías existentes, como Microsoft SQL Server y ADO.NET. Para simplificar la programación de aplicaciones, WPF proporciona un motor de enlace de datos que realiza estos pasos automáticamente. La unidad que constituye el núcleo del motor de enlace de datos es la clase **Binding**, encargada de enlazar un control (el destino del enlace) a un objeto de datos (el origen del enlace).

El motor de enlace de datos de WPF proporciona compatibilidad adicional que incluye validación, ordenación, filtrado y agrupación. Además, el enlace de datos admite el uso de plantillas de datos, a fin de crear una interfaz de usuario personalizada para los datos enlazados cuando la interfaz de usuario mostrada por los controles estándar de WPF no es adecuada.

Gráficos

WPF presenta un conjunto extenso, escalable y flexible de características de gráficos que aportan las ventajas siguientes:

- Gráficos independientes de la resolución e independientes del dispositivo. La unidad de medida básica del sistema de gráficos de WPF es el píxel independiente del dispositivo, que es 1/96 de pulgada, independientemente de la resolución de pantalla real, y que proporciona la base para la representación independiente de la resolución y del dispositivo. Cada píxel independiente del dispositivo se escala automáticamente para coincidir con el valor de puntos por pulgada (ppp) del sistema en que se representa.
- Precisión mejorada. El sistema de coordenadas de WPF se mide con números de punto flotante de precisión doble, en lugar de precisión simple. Las transformaciones y los valores de opacidad también se expresan como doble precisión. WPF admite



también una amplia gama de colores (scRGB) y proporciona compatibilidad integrada para administrar las entradas desde espacios de color diferentes.

- Compatibilidad con gráficos avanzados y animación. WPF simplifica la programación de gráficos administrando automáticamente las escenas de animación.
- Aceleración de hardware. El sistema de gráficos de WPF saca partido del hardware de gráficos para minimizar el uso de CPU.

Formas 2D

WPF proporciona una biblioteca de formas 2D comunes dibujadas mediante vectores. Una función interesante de las formas es que no sirven únicamente para su presentación; las formas implementan muchas de las características que cabe esperar de los controles, incluida la entrada de datos desde el teclado y el Mouse.

Geometrías 2D

Las formas 2D proporcionadas por WPF abarcan el conjunto estándar de formas básicas. Sin embargo, puede que sea preciso crear formas personalizadas para facilitar el diseño de una interfaz de usuario personalizada. Para este fin, WPF proporciona las geometrías. Estas permiten dibujar directamente, utilizar como un pincel o utilizar para recortar otras formas y controles.

Los objetos **Path** se pueden utilizar para dibujar formas cerradas o abiertas, varias formas o incluso formas curvas.

Los objetos **Geometry** se pueden utilizar para el recorte, la comprobación de visitas y la representación de datos de gráficos 2D.

Efectos 2D

Un subconjunto de las funciones 2D de WPF son los efectos visuales, tales como degradados, mapas de bits, dibujos, pintar con vídeos, rotación, ajuste de escala y sesgo. Todas ellas se aplican mediante pinceles.

Representaciones 3D

WPF también incluye funciones de representación 3D que se integran con los gráficos 2D para permitir la creación de interfases de usuarios más importantes.

Animaciones

La compatibilidad de WPF con la animación permite hacer que los controles crezcan, tiemblen, giren o se desvanezcan, crear transiciones de página interesante, y mucho más. Puede animar la mayoría de las clases de WPF, incluso las personalizadas.

Multimedia

Una manera de mostrar un contenido enriquecido es utilizar medios audiovisuales (multimedia). WPF proporciona compatibilidad especial con imágenes, vídeo y audio.

- Imágenes: Las imágenes están presentes en la mayoría de las aplicaciones y WPF proporciona varias maneras de utilizarlas.
- Vídeo y Audio: El control **MediaElement** es capaz de reproducir vídeo y audio y presenta la flexibilidad suficiente para constituir la base de un reproductor multimedia personalizado. El marcado XAML siguiente implementa un reproductor multimedia.



Texto y Tipografía

Para facilitar una representación de texto de gran calidad, WPF ofrece las características siguientes:

- Compatibilidad con fuentes OpenType.
- Mejoras de ClearType.
- Alto rendimiento que saca partido de la aceleración de hardware.
- Integración de texto con multimedia, gráficos y animación.
- Compatibilidad con fuentes internacionales y mecanismos de reserva.

Documentos

WPF permite trabajar de forma nativa con tres tipos de documentos: documentos dinámicos, documentos fijos y documentos de XML Paper Specification (XPS). WPF proporciona también servicios para crear, ver, administrar, anotar, empaquetar e imprimir documentos.

- Documentos dinámicos: Los documentos dinámicos se han diseñado para optimizar su presentación y legibilidad ajustando dinámicamente su contenido y modificando su flujo cuando se producen cambios en el tamaño de la ventana y la configuración de pantalla.
- Documentos Fijos: Los documentos fijos están destinados a aplicaciones que requieren una presentación con representación fiel (lo que se ve es lo que se obtiene, o WYSIWYG, en sus siglas en inglés) precisa, especialmente por lo que respecta a su impresión. Los usos típicos para los documentos fijos incluyen la creación de publicaciones, el procesamiento de textos y el diseño de formularios, donde es vital que se respete el diseño de página original. Los documentos fijos conservan la organización precisa de su contenido de una manera independiente del dispositivo. El diseño permanece inalterado en todos los casos, aunque la calidad del documento varía según las funciones de cada dispositivo.
- Documentos XPS: Los documentos de XML Paper Specification (XPS) se basan en documentos fijos de WPF. Los documentos de XPS se describen con un esquema basado en XML que básicamente es una representación en forma de página de documentos electrónicos. XPS es un formato de documento abierto habilitado para varios exploradores y diseñado para facilitar la creación, el uso compartido, la impresión y el almacenamiento de documentos paginados.

Anotaciones

Las anotaciones son notas o comentarios que se agregan a los documentos para marcar la información o resaltar elementos de interés, a fin de consultarlos más adelante. Aunque escribir notas en documentos impresos es fácil, la posibilidad de "escribir" notas en los documentos electrónicos con frecuencia es limitada o no está disponible en absoluto. Sin embargo, en WPF se proporciona un sistema de anotaciones que admite la inserción de notas rápidas y el resaltado.

Empaquetado

Las **APISystem.IO.Packaging** de WPF permiten que las aplicaciones organicen los datos, el contenido y los recursos en un único documento ZIP portátil, sencillo de distribuir y de fácil acceso. Es posible incluir firmas digitales para autenticar los elementos contenidos en un paquete y comprobar que el elemento firmado no se haya manipulado ni modificado. También puede cifrar los paquetes mediante la administración de derechos para restringir el acceso a la información protegida.

Impresión



Microsoft .NET Framework incluye un subsistema de impresión al que WPF aporta, además, compatibilidad con el control de sistemas de impresión mejorados. Las mejoras de impresión incluyen las siguientes:

- Instalación en tiempo real de servidores de impresión y colas remotos.
- Detección dinámica de funciones de impresora.
- Configuración dinámica de opciones de impresión.
- Modificación del enrutamiento y las prioridades de los trabajos de impresión.

Los documentos XPS presenta, además, una mejora fundamental del rendimiento. La ruta de acceso de Interfaz de dispositivo gráfico de Microsoft Windows (GDI) de impresión suele requerir dos conversiones:

- La primera conversión del documento a un formato de procesador de impresión, como Metarchivo mejorado (EMF).
- Una segunda conversión al lenguaje de descripción de páginas de la impresora, como PCL o PostScript.

Sin embargo, los documentos XPS evitan estas conversiones porque un componente del formato de archivo XPS es un lenguaje de procesador de impresión y un lenguaje de descripción de páginas. Esta compatibilidad ayuda a reducir el tamaño del archivo de cola y las cargas de las impresoras integradas en red.

Personalización

Para simplificar la organización de los controles en una interfase de usuario y asegurarse de que la organización se conserve aunque se modifiquen el tamaño de la ventana y la configuración de pantalla, se utiliza el sistema de diseño de WPF. Dado que la mayoría de las aplicaciones permiten a los usuarios interactuar con los datos, los enlaces de datos se utilizan para reducir el trabajo de integración de la interfaz de usuario con esos datos. A fin de mejorar la apariencia visual de la aplicación, se utiliza toda la gama de gráficos, animación y multimedia que WPF proporciona. Por último, si la aplicación funciona a través de texto y documentos, puede utilizar las funciones de tipografía, documentos, anotación, empaquetado e impresión de WPF.

Sin embargo, con frecuencia estos elementos fundamentales no bastan para crear y administrar una experiencia del usuario realmente diferenciada y visualmente impactante. Puede que los controles de WPF no se integren con la apariencia deseada de la aplicación. Es posible que los datos no se muestren del modo más eficaz. La apariencia y el funcionamiento predeterminados de los temas de Windows pueden no ser adecuados para proporcionar la experiencia global del usuario con respecto a la aplicación. En muchos aspectos, una tecnología de presentación requiere la extensibilidad visual tanto como cualquier otro tipo de extensibilidad.

Por esta razón, WPF proporciona gran variedad de mecanismos para la creación de experiencias de usuario únicas, incluido un modelo de contenido enriquecido para los controles, estilos, desencadenadores, plantillas de controles y datos, estilos, recursos de la interfaz de usuario, temas y máscaras.

- **Modelo de Contenido:** El propósito principal de la mayoría de los controles de WPF es mostrar contenido. En WPF, el tipo y número de elementos que pueden constituir el contenido de un control se denomina el modelo de contenido del control. Algunos controles pueden contener un solo elemento y tipo de contenido; por ejemplo, el contenido de un control *TextBox* es un valor de tipo String que está asignado a la propiedad *Text*. Otros controles, sin embargo, pueden contener varios elementos con tipos diferentes de contenido; el contenido de un control *Button*, especificado por la propiedad *Content*, puede contener gran variedad de elementos, entre los que se incluyen controles de diseño, texto, imágenes y formas.
- **Desencadenadores (Triggers):** Aunque el propósito principal del marcado XAML es implementar la apariencia de una aplicación, también puede utilizar XAML para implementar algunos aspectos del comportamiento de una aplicación. Un ejemplo de



ello es el uso de desencadenadores para cambiar la apariencia de una aplicación de acuerdo con las interacciones con el usuario.

- Plantillas de Control: Las UIs predeterminadas para los controles de WPF suelen construirse a partir de otros controles y formas. A veces, la apariencia predeterminada de un control puede ser incongruente con la apariencia global de una aplicación. En este caso, puede utilizar una plantilla de control **ControlTemplate** para cambiar la apariencia de la interfase de usuario del control sin modificar su contenido ni su comportamiento.
- Plantilla de Datos: Mientras que una plantilla de control permite especificar la apariencia de un control, una plantilla de datos permite especificar la apariencia del contenido del control. Las plantillas de datos se utilizan con frecuencia para mejorar la manera de mostrar los datos enlazados.
- Estilos: Los estilos permiten a los programadores y diseñadores normalizar un aspecto determinado de sus productos. WPF proporciona un modelo de estilo sólido, que es la base del elemento **Style**.
- Recursos: Los controles de una aplicación deben tener la misma apariencia, que puede incluir todo tipo de recursos, desde fuentes y colores de fondo hasta plantillas de control, pasando por las plantillas de datos y los estilos. Puede utilizar la compatibilidad de WPF con los recursos de la interfase de usuario (UI) para encapsular estos recursos en una ubicación única y poder reutilizarlos.
- Temas y Máscaras: Desde una perspectiva visual, un tema define la apariencia global de Windows y de las aplicaciones que se ejecutan dentro del mismo. La apariencia definida por un tema establece la apariencia predeterminada de una aplicación WPF. Sin embargo, WPF no se integra directamente con los temas de Windows. Como la apariencia de WPF se define mediante plantillas, WPF incluye una plantilla para cada uno de los temas conocidos de Windows, como Aero (Windows Vista), Clásico (Microsoft Windows 2000), etc. Estos temas están empaquetados en diccionarios de recursos que se resuelven cuando no se encuentran los recursos correspondientes en una aplicación. Por otro lado, la experiencia del usuario para algunas aplicaciones no procede necesariamente de los temas estándar. Tales UIs tienden a proporcionar temas personalizados, específicos de cada aplicación. Se denominan máscaras, y las aplicaciones con máscaras suelen proporcionar enlaces que permiten a los usuarios personalizar diversos aspectos de la máscara.
- Controles Personalizados: Aunque WPF proporciona una amplísima compatibilidad con funciones de personalización, puede encontrar situaciones en que los controles existentes de WPF no satisfagan las necesidades de la aplicación o de los usuarios. WPF permite crear controles.

3- Eventos y Comandos

Información general sobre Eventos Enrutados

En los primeros pasos con WPF, lo más probable es que use eventos enrutados aún sin saber que los está usando. Por ejemplo, si agrega un botón a una ventana en el diseñador de Visual Studio®, modifica el nombre a miBoton y después hacer doble clic sobre él, el evento *Click* se conectará al marcado XAML y se agregará un controlador de eventos para este evento al código que hay tras la clase *Window*. Esto es lo mismo que la conexión de eventos en Windows Forms y ASP.NET. En concreto, en el marcado XAML para el botón, terminará por tener un código parecido a este:

```
<Button Name=" miBoton " Click="miBoton_Click">Click Me</Button>
```



La declaración de XAML para incorporar un evento es parecida a una asignación de propiedad en XAML, pero el resultado es un enlace de evento normal en el objeto que especificó el controlador de eventos. Si cambia la ventana al editor de código verá lo siguiente:

```
private void myButton_Click(object sender, RoutedEventArgs e) { }
```

Esto es igual a cualquier otra conexión de evento .NET, se tiene un delegado explícitamente declarado y enlazado a un evento de un objeto, y el delegado apunta a un método de control. El único indicio de que se están usando eventos enrutados es el tipo del argumento de evento para el evento *Click*, que es **RoutedEventArgs**. Entonces, ¿qué tienen de especial los eventos enrutados?

Un evento enrutado es un tipo de evento que puede invocar controladores o varios agentes de escucha en un árbol de elementos, en lugar de simplemente en el objeto que lo desencadenó. Para comprender mejor esta definición, necesitará antes comprender el modelo de composición elemental de WPF.

Árboles de elementos WPF

Si empieza con una ventana nueva en un proyecto y arrastra un botón en la ventana del diseñador, terminará con un árbol de elementos en el XAML parecido a éste (los atributos se omitieron para mejorar la claridad):

```
<Window>
  <Grid>
    <Button/>
  </Grid>
</Window>
```

Cada uno de estos elementos representa una instancia en tiempo de ejecución de un tipo .NET correspondiente y la jerarquía declarada de elementos forma lo que se denomina un árbol lógico. Además, muchos controles de WPF son a su vez contenedores, lo que significa que pueden tener elementos secundarios. Por ejemplo, un *Button* puede tener un elemento secundario complejo como contenido. El árbol lógico se podría expandir como se muestra aquí:

```
<Window>
  <Grid>
    <Button>
      <StackPanel>
        <Image/>
        <TextBlock/>
      </StackPanel>
    </Button>
  </Grid>
</Window>
```

Como puede imaginar, el árbol podría contener varias ramas y el árbol lógico puede crecer considerablemente en complejidad. Lo más importante a tener en cuenta sobre los elementos WPF del árbol lógico es que lo que se ve no es realmente lo que se obtiene en tiempo de ejecución. Cada uno de esos elementos suele ampliarse hasta formar un árbol más complejo de elementos visuales en tiempo de ejecución.

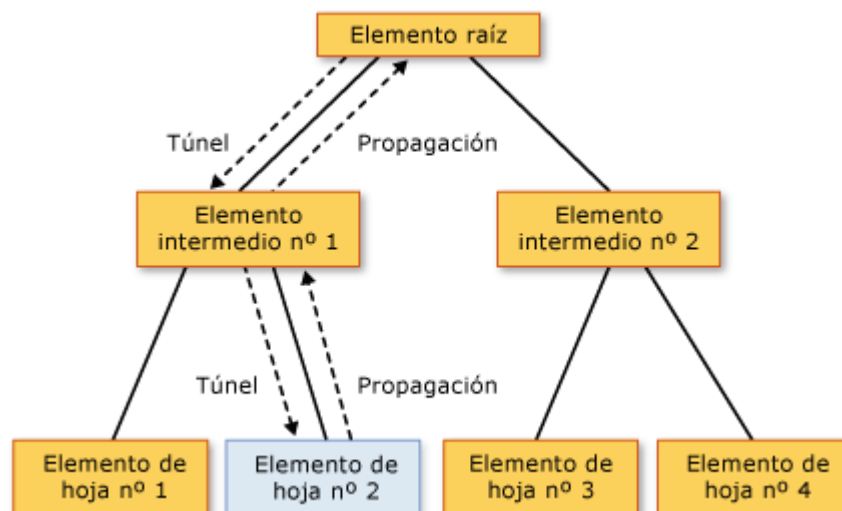
Cuando hago clic en mi botón, puede que en realidad no esté haciendo clic en el elemento *Button*; puedo estar haciendo clic en un elemento secundario del árbol visual, posiblemente incluso uno que no se muestre en mi árbol lógico (como *ButtonChrome*). Por ejemplo, digamos que hago clic sobre la imagen que contiene mi botón. En realidad, el clic se manifiesta

inicialmente como un evento de *MouseDownLeftButton* incluido dentro del elemento *Image*. Pero de alguna forma esto hay que trasladarlo a un evento *Click* al nivel del *Button*. Es aquí donde entra en juego el enrutamiento en eventos.

Enrutamiento de eventos

Es importante entender un poco los árboles lógicos y visuales porque los eventos enrutados se enrután principalmente según el árbol visual. Los eventos enrutados admiten una **RoutingStrategy** que puede ser *Bubble*, *Tunnel* o *Direct*.

- **Bubble** es la más común y significa que un evento se propagará hacia la parte superior del árbol visual a partir del elemento origen hasta que se controle o alcance el elemento raíz. Esto le permite controlar un evento de un objeto aún más arriba de la jerarquía de elementos desde el elemento origen. Por ejemplo, se podría adjuntar un controlador *Button.Click* en elemento *Grid* que lo contiene en vez de directamente en el propio botón.
- **Tunnel**: Los eventos *Tunnel* van en dirección contraria, se inician en el elemento raíz y van bajando por el árbol de elementos hasta que se controlan o alcanzan el elemento origen para el evento. Esto permite que los elementos precedentes intercepten el evento y lo controlen antes de que alcance el elemento origen. Los eventos *Tunnel* usan como prefijo de sus nombres *Preview* como convención (por ejemplo, *PreviewMouseDown*).
- **Direct**: Los eventos directos se comportan como eventos normales de .NET Framework. El único controlador posible para el evento es un delegado que se enlaza al evento.



Eventos enrutados y composición

Vayamos paso a paso para ver cómo el evento *Button.Click* se produce y regresa a casa, ya que es importante. Como he mencionado anteriormente, un usuario iniciará un evento *Click* con un evento *MouseDownLeftButton* en algún elemento secundario del árbol visual del *Button*, como en el caso de *Image* en el ejemplo anterior.

Cuando se produce el evento *MouseDownLeftButton* dentro del elemento *Image*, se inicia un evento *PreviewMouseDownLeftButton* en la raíz y profundiza hacia *Image*. Si ninguno de los controladores establecen la marca *Handled* en verdadero para el evento de vista previa, se inicia la propagación del evento *MouseDownLeftButton* desde el elemento *Image* hasta que



llega a *Button*. El botón controla ese evento, establece la marca *Handled* en verdadero y acciona su propio evento *Click*.

Las implicaciones son bastante eficaces. Por ejemplo, si elijo reemplazar la apariencia predeterminada del botón aplicando una plantilla de control que contenga un elemento *Ellipse*, no tengo que hacer nada para garantizar que los clics hechos fuera de *Ellipse* no accionen el evento *Click*. Los clics que quedan justo fuera del borde de *Ellipse* seguirán estando dentro de los límites rectangulares de mi botón, pero *Ellipse* tiene su propia detección de aciertos para *MouseLeftButtonDown*, cosa que no ocurre para las partes vacías del botón que están fuera de *Ellipse*.

Así que sólo los clics que se hacen dentro de *Ellipse* accionan el evento *MouseLeftButtonDown*. Sigue estando controlado por la clase *Button* a la que se adjunta la plantilla, así que se obtendrá el comportamiento previsible incluso del botón personalizado. Esto también es un concepto muy importante que recordar al escribir sus propios controles compuestos personalizados, porque lo más probable es que necesite hacer cosas parecidas a las que hace *Button* para administrar eventos de elementos secundarios que se coloquen dentro de su control.

Eventos adjuntos

Los eventos adjuntos son eventos enrutados que admiten un enlace XAML sobre elementos distintos del tipo en el que se declara el evento. Por ejemplo, si desea que el elemento *Grid* esté atento al paso de un evento *Button.Click*, sólo tendría que enlazarlo de esta forma:

```
<Grid Button.Click="myButton_Click">
    <Button Name="myButton" >Presionar</Button>
</Grid>
```

Los eventos adjuntos sólo dan un poco más de flexibilidad en los sitios donde conecta sus controladores de eventos. Pero si los elementos están contenidos en la misma clase (como en este ejemplo), puede no resultar aparente qué diferencia hay porque, en ambos casos, el método de control sigue siendo sólo un método de la clase *Window*.

Información general sobre Comandos enrutados

Los comandos enrutados de WPF ofrecen un mecanismo específico para enlazar controles de IU como botones de barra de herramientas y elementos de menú a controladores, sin tener que realizar muchos emparejamientos integrados ni código repetitivo en la aplicación. Los comandos enrutados ofrecen tres cosas importantes además del control normal de eventos:

- Los elementos origen del comando enrutado (invocadores) pueden estar desconectados de los destinos de comandos (controladores), no es necesario que haya referencias directas entre ellos, como ocurriría si estuvieran vinculados por un controlador de eventos.
- Los comandos enrutados habilitarán o deshabilitarán todos los controles de IU asociados cuando el controlador indica que el comando está deshabilitado.
- Los comandos enrutados le permiten asociar métodos abreviados de teclado y otras formas de movimientos de entrada (movimientos de lápiz óptico, por ejemplo) como otra manera de invocar el comando.

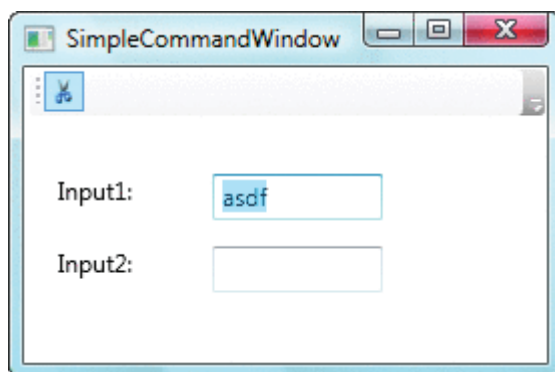
Además, una variedad específica del comando enrutado (la clase **RoutedUICommand**) agrega la capacidad de definir una sola propiedad *Text* para usarla como indicaciones de texto de cualquier control que sea invocador para el comando. La propiedad *Text* también se puede localizar más fácilmente que visitando todos los controles del invocador asociado.

Para declarar un comando en un invocador, sólo tiene que establecer una propiedad **Command** sobre el control que iniciará el comando:

```
<Button Command="ApplicationCommands.Save">Guardar</Button>
```

La propiedad *Command* la admiten *MenuItem*, *Button*, *RadioButton*, *Checkbox*, *Hyperlink* y otros controles.

Para concretar y ver rápidamente las ventajas de los comandos enrutados, veamos un ejemplo sencillo. En la siguiente figura se puede ver una IU con dos TextBoxes y un botón de barra de herramientas para realizar una acción Cortar sobre el texto de los cuadros de texto.



Para poder conectar esto usando eventos, necesitaría definir un controlador *Click* para el botón de la barra de herramientas y ese código necesitaría hacer referencia a los dos cuadros de texto. Tendría que determinar qué cuadro de texto tiene el foco y llamar a operaciones apropiadas del portapapeles según la selección del texto en el control. También tendría que preocuparse de habilitar y deshabilitar el botón de la barra de herramientas según donde esté el foco y si se seleccionó algo en el cuadro de texto. Esto generaría un código complejo y desordenado.

Con comandos, lo único que hay que hacer es establecer la propiedad **Command** del botón de la barra de herramientas en el comando **ApplicationCommands.Cut** que está definido en WPF y ya está listo.

```
<StackPanel>
  <ToolBar DockPanel.Dock="Top" Height="25">
    <Button Command="ApplicationCommands.Cut">
      <Image Source="cut.ico"/>
    </Button>
  </ToolBar>
  <TextBox Name="txt1" Width="100" Height="23"></TextBox>
  <TextBox Name="txt2" Width="100" Height="23"></TextBox>
</StackPanel>
```

Ahora podría ejecutar la aplicación y comprobar que el botón de la barra de herramientas está inicialmente deshabilitado. Después de seleccionar texto en uno de los cuadros de texto, el botón de la barra de herramientas pasa a estar habilitado y, si se hace clic en él, el texto se corta y pasa al portapapeles. Y funcionaría para cualquier TextBox de cualquier lugar de la IU. Lo que sucede aquí es que la implementación de la clase TextBox tiene un enlace de comandos integrado para el comando Cut y encapsula automáticamente el control del portapapeles para ese comando (además de Copy y Paste). Pero, ¿cómo se invoca solamente el comando en el cuadro de texto enfocado, y cómo llega el mensaje al cuadro de texto para decirle que controle el comando? Aquí es donde entra en juego el enrutamiento de comandos.



Enrutamiento de comandos

La diferencia entre comandos enrutados y eventos enrutados es la forma en que el comando se dirige desde el invocador del comando al controlador del comando. Concretamente, los eventos enrutados se usan internamente para dirigir mensajes entre los invocadores y los controladores de comandos (mediante el enlace de comandos que se engancha en el árbol visual).

Aquí podría haber una relación de varios a varios, pero sólo un controlador de comando estará realmente activo en cada momento. El controlador de comandos activo lo determina una combinación de las posiciones del invocador y del administrador de comandos en el árbol visual, y de la posición del foco en la interfase de usuario. Los eventos enrutados se usan para llamar al controlador de comandos activo para preguntarle si se debe habilitar el comando, así como para invocar al controlador de método *Execute* del controlador de comandos.

Generalmente, un invocador de comandos busca un enlace de comandos entre su propia ubicación en el árbol visual y la raíz del árbol visual. Si encuentra uno, el controlador de comandos enlazado determinará si el comando está habilitado y se llamará cuando se invoque el comando. Si el comando está conectado a un control de una barra de herramientas o de un menú, entonces se ejecuta lógica adicional que también examina la ruta de acceso del árbol visual desde la raíz al elemento del foco buscando un enlace de comando.

Definición de comandos

Existen cinco clases de comandos integradas en WPF:

- **ApplicationCommands:** Close, Cut, Copy, Paste, Save, Print
- **NavigationCommands:** BrowseForward, BrowseBack, Zoom, Search
- **EditingCommands:** AlignXXX, MoveXXX, SelectXXX
- **MediaCommands:** Play, Pause, NextTrack, IncreaseVolume, Record, Stop
- **ComponentCommands:** MoveXXX, SelectXXX, ScrollXXX, ExtendSelectionXXX

XXX indica diversas operaciones como MoveNext y MovePrevious. Los comandos de cada clase están definidos como propiedades para que pueda conectarlos fácilmente.

También puede usar estos comandos con una notación abreviada

```
<Button Command="Save">Save</Button>
```

Desafíos de los comandos enrutados

Los comandos enrutados funcionan bien en escenarios de interfases de usuario sencillas, conectando barras de herramientas y elementos de menú, y controlando esas cosas que están emparejadas intrínsecamente al foco del teclado (como las operaciones con el portapapeles). Sin embargo, donde los comandos enrutados se muestran insuficientes, es cuando se empieza a crear interfases de usuario complejas, donde la lógica del control de comando está en el código de soporte para las definiciones de vista y los invocadores de comando no siempre están dentro de una barra de herramientas o de un menú.

El problema que surge al trabajar de este modo es que la habilitación y el control de lógica para un comando puede no formar parte del árbol visual directamente; en su lugar puede estar situada en un presentador o en un modelo de presentación. Además, el estado que determina si el comando debe habilitarse puede no tener relación con la situación de los invocadores de comandos y las vistas en el árbol visual. También puede encontrar casos donde haya más de un controlador válido al mismo tiempo para un comando particular.

Para evitar los problemas potenciales de la ubicación del árbol visual con comandos enrutados, necesitará simplificar las cosas. Normalmente necesitará asegurarse de que los controladores de comando están en el mismo elemento o en niveles superiores del árbol visual del elemento que invocará el comando.